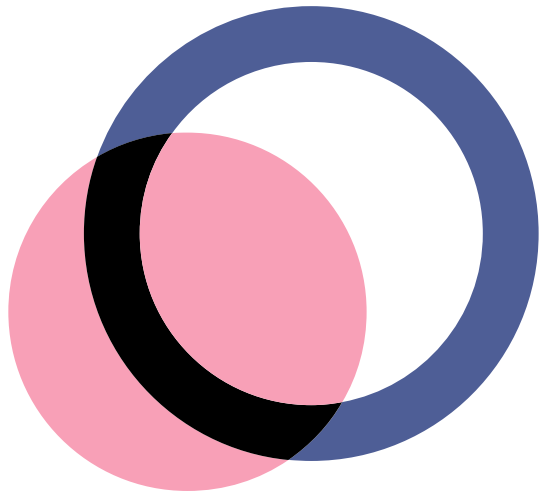
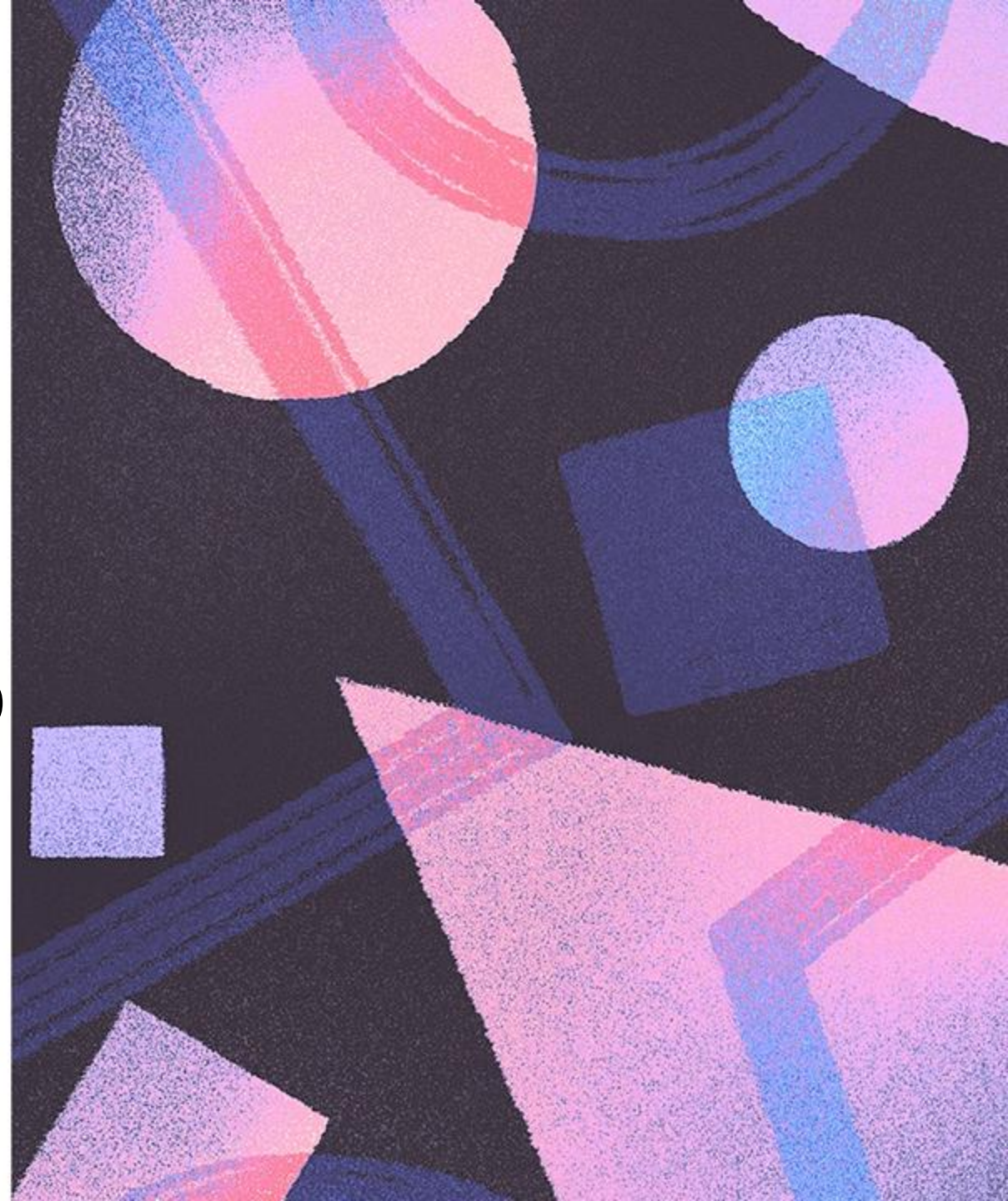




Design Patterns Lab

Lecture 8

> Command Pattern



Command Pattern

- Part of Behavioral Design Patterns (BDP)
- Focus: Encapsulate requests/actions as objects
- Core Idea: Decouple the object requesting an action from the object performing it
- Enables:
 - Queuing operations
 - Undo/redo
 - Dynamic action execution

Problem Statement

- Systems often tightly couple requesters and executors
- Example: Button directly calls device method → rigid, hard to extend
- Challenges:
 - Changing actions requires modifying many classes
 - Queuing and storing actions is difficult
 - Undo/redo is almost impossible

Command Pattern Core Idea

- Every action is represented as a Command object
- Invoker triggers commands without knowing details
- Receiver performs actual work
- Commands wrap requests and delegate execution to receivers

Components of Command Pattern

- Command Interface — defines execute()
- Concrete Commands — specific actions; store receiver
- Receiver — executes the actual operation
- Invoker — triggers command execution

Receiver

- Role: Performs real work
- Knows how to perform tasks, but not who requested them
- Example: Light class → turn on/off

Command Objects

- Represent executable requests
- Store reference to receiver
- Delegates execution to receiver
- Interchangeable and reusable at runtime

Invoker

- Triggers commands
- Does not know details of action or receiver
- Stores command reference and calls execute()
- Allows dynamic replacement of commands

Execution Flow

Steps:

1. Client creates receiver
2. Client creates command object(s) linking to receiver
3. Command assigned to invoker
4. Invoker executes command → receiver acts
5. Commands can be replaced or queued dynamically

Key Advantages

- Loose coupling between sender and receiver
- Extensible: add new commands without modifying existing code
- Reusable requests
- Supports undo/redo and macro commands
- Enables scheduling and queuing

When to Use

Use Command Pattern when:

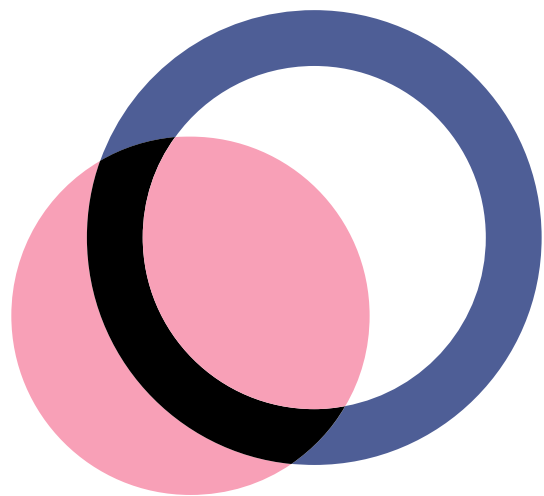
- Actions should become reusable objects
- Sender and receiver should be decoupled
- Requests need queuing or scheduling
- Undo/redo is required
- Dynamic runtime behavior is desired

When to Avoid

- Small/simple systems → extra abstraction may be unnecessary
- Adds classes and objects → complexity overhead

Activity: Smart Office Automation System

- A smart office system controls Lights, Projector, AC, and Coffee Machine
- The control panel triggers commands without knowing device details
- Design commands for each device and implement the invoker
- Goal: test runtime flexibility, command swapping, and optional undo/macro commands



**THANK
YOU**

